

Boutique Hotel Technikum

A full-stack hotel booking app

Group AE · Hotel Mario · Advanced Web Technologies · MSE
Artner Patrick, Auß Michael, Bohrn Johannes, Neirukh Jusef

Ionic Vue 7

Spring Boot 4

MySQL 8

Clean Architecture

Pinia

Containers

Live Demo

Walk the user journey end-to-end (~5 min)

1 Landing Page

Home screen — hotel overview, slideshow, CTA button

2 Room Selection

Filter by date range (two-step date picker), price, available-only toggle

3 Pagination

Step through pages — server-side, 5 rooms per page

4 Book a Room

Select available room → form with validation → review screen

5 Confirmation

Show confirmation card: booking ID, room image, hotel details, print to A4

6 Error path

Attempt to book an already-taken room → 409 'Room no longer available'

AI Workflow

Clean Architecture Scaffolding from API Spec · ~5 min

What we gave AI

- ▶ Our ER diagram (ROOM, EXTRA, BOOKING, ROOM_EXTRA)
- ▶ The OpenAPI spec we wrote (endpoints, DTOs, pagination shape)
- ▶ Architecture decision: component packages, v1 versioning, interface/impl split, Lombok

What AI generated (one pass)

- ▶ Entities, JPA repositories, service interfaces + impls, mappers
- ▶ @EmailsMatch and @CheckOutAfterCheckIn as proper class-level constraint annotations
- ▶ GlobalExceptionHandler returning structured {status, message, errors[]}
- ▶ 13 E2E tests (Testcontainers + MockMvc) covering overlap, 409, pagination

What we changed

Minor corrections only — one wrong Spring Boot 4 import, one overly broad exception catch.

Example: cross-field constraint annotation

```
@EmailsMatch // class-level
@CheckOutAfterCheckIn // class-level
@Data @Builder @NoArgsConstructor
public class BookingRequestDto {
    @NotBlank @Email String email;
    @NotBlank String emailConfirmation;
    @FutureOrPresent LocalDate checkIn;
    LocalDate checkOut;

    // ...
}

// Validator generated correctly first attempt:
public class EmailsMatchValidator implements
    ConstraintValidator<EmailsMatch, BookingRequestDto> {
    public boolean isValid(BookingRequestDto dto, ...) {
        return
            dto.getEmail().equals(dto.getEmailConfirmation());
    }
}
```

Human decision: we defined the architecture and validation rules — AI filled in the boilerplate correctly.

What Surprised Us

One thing easier than expected · one thing it got wrong

✓ Easier than expected

Full-stack scaffolding from architecture doc

We handed AI our ER diagram and API spec. It generated the entire clean-architecture backend skeleton — entities, repos, services, mappers, DTOs, controllers — in one pass. Only minor corrections needed.

Cross-field validation annotations

@EmailsMatch and @CheckOutAfterCheckIn as proper class-level constraint annotations with validator classes — correct on the first attempt. Would have taken us 30 min to research and write manually.

Test coverage from specs

Given endpoint descriptions, AI wrote 13 meaningful E2E tests (Testcontainers + MockMvc) including overlap checks and 409 conflict cases we hadn't thought of.

✗ Where it led us astray

Fake pagination (peer review catch)

AI generated fetchRooms() hardcoded to getRooms(0, 20) — all rooms, no real pages. Pagination buttons existed but were cosmetic. Only caught during peer review. Root cause: we didn't specify 'real server-side pagination' explicitly.

Wrong TypeScript fix loops

The DatetimeHighlightStyle type error: AI tried adding border: 'none' (wrong property) instead of removing the return type annotation. Took two prompts to converge because we didn't explain the union type constraint upfront.

imageUrl mismatch

Backend DataSeeder used filename-based paths (/images/rooms/standard.jpg) but the actual public files are numbered (/images/rooms/1.jpg). AI never caught this — we found it visually. Lesson: AI can't verify runtime asset resolution.

Architecture at a Glance

Clean Architecture · Atomic Design · Pinia stores

Backend — Spring Boot 4, Java

config/	HotelConfig, DataSeeder, WebConfig
common/	GlobalExceptionHandler, PageResult, exceptions
components/rooms/	api/v1 → service → repository → model
components/bookings/	api/v1 → service → repository → model
api/dtos/v1/	RoomDto, BookingRequestDto, BookingConfirmationDto ...
api/mapper/v1/	RoomMapper, BookingMapper (interface + impl)
service/	RoomService, RoomAvailabilityService (interface + impl)
repository/	JPA: RoomRepository, RoomAvailabilityRepository

Frontend — Ionic & Vue, TypeScript

views/	RoomSelectionView, BookingView, BookingReviewView, BookingDetailsView
components/atoms/	ExtraChip, AvailabilityBadge, DetailRow
components/molecules/	RoomCard, BookingForm, DatePickerModal, ImageSlider, SharedHeader/Footer
components/organisms/	RoomList, FilterBar
stores/	useRoomStore (rooms, pagination), useFilterStore (dates, price), useBookingStore
services/api.ts	Typed DTOs, getRooms(), checkAvailability(), bookRoom()
router/	Vue Router — /home, /room, /booking/:id, /booking/details

Peer Review 2 — Feedback & Response

Component complexity · code duplication · atomic design

What the reviewer flagged

- ▶ Duplicated template logic across views (same label/value rows repeated by hand)
- ▶ Duplicated slideshow code in home.vue and about.vue (~80 lines each)
- ▶ No consistent reuse of existing atoms (ExtraChip used in RoomCard but not in confirmation view)

What we changed (with AI)

atoms/DetailRow.vue

New atom: `<strong>Label:</strong> Value` row — replaced 16 ion-item blocks across BookingDetailsView + BookingReviewView

molecules/ImageSlider.vue

New molecule: reusable slideshow — removed ~160 lines of duplicated script + template from home.vue and about.vue

ExtraChip (extended)

Applied existing atom in BookingDetailsView to match RoomCard — consistent chip rendering throughout

Atomic Design structure (after)

atoms/ ExtraChip · AvailabilityBadge · DetailRow

molecules/ RoomCard · BookingForm · DatePickerModal
ImageSlider · SharedHeader · SharedFooter

organisms/ RoomList · FilterBar

What we scoped out (not actioned)

- ▶ HotelConfig → HotelDto cross-layer dep. — non-functional, deferred
- ▶ FilterBar untyped props — low priority, no TypeScript errors

Human decision: scoped to high-impact duplications only — explicit choice not to over-refactor

Questions?

One question per team, consider:

- AI usage
- architecture
- things that went wrong